

CHP vs stabilizer-python: tableau output and performance

This report compares **Aaronson–Gottesman stabilizer tableaux** between the reference **CHP** simulator ([learning_material/CHP/chp.c](#)) and this repo’s **Python** package ([stabilizer-python/stabilizer_python/](#)). It also includes **measurement-outcome samples**, **plots**, and **performance metrics** from the benchmark harness.

How to regenerate performance assets and summaries

```
python .\reports\chp_vs_stabilizer_benchmark.py --n 200 --warmup 10
```

Outputs (figures + JSON + short markdown summary) are written under [reports/chp_vs_stabilizer_assets/](#).

CHP build (reference)

```
cd learning_material\CHP
gcc -o chp.exe chp.c
```

Python tests

```
cd stabilizer-python
python -m pytest -q
```

1. Tableau and “matrix” output (methodology)

What CHP prints

With the `-q` flag, CHP prints a banner **Final state:** then calls `printstate` **twice**, with `gaussian()` in between (see `runprog` in `chp.c`). The first block is the raw tableau after the circuit; the second block is an equivalent tableau after internal Gaussian elimination on the stabilizer generators. **Pauli letters (I/X/Y/Z) and the x/z bit tables are the same object** CHP uses internally; it does not print separate numeric X/Z matrices to the console.

Each row is a **destabilizer** (first `n` rows) or **stabilizer** (last `n` rows), prefixed with `+` or `-` from CHP’s phase vector `r` (mod 4).

What stabilizer-python prints

`StabilizerState` exposes:

- `format_chp_printstate()` — same row layout as CHP’s `printstate` (separator line between destabilizer and stabilizer blocks).
- `format_xz_binary_matrices()` — explicit **X matrix** and **Z matrix** (each $2n \times n$, entries 0/1), matching CHP’s internal bit layout.

Python tracks only a **binary sign** per row (± 1 on the row’s Pauli product), not full $\pm i$ phases, so **+/- prefixes may differ** from CHP when CHP’s `r` is 1 or 3 (imaginary overall phase on a row). The **Pauli pattern** (I/X/Y/Z per qubit) should still match CHP’s **first printstate** block when the same Clifford sequence is applied.

Small circuits used only for tableau comparison

These files stop **before** measurements so the quantum state is pure and identical runs agree line-for-line on the Pauli tableau (first CHP block):

Snapshot	CHP file	Role
EPR Bell prep	reports/chp_vs_stabilizer_assets/circuits_tableau/epr_bell_prep.chp	H(q0) then CNOT(q0→q1) on <code>\ 00</code>
GHZ prep (prefix of ghz.chp)	reports/chp_vs_stabilizer_assets/circuits_tableau/ghz_prep.chp	Same Clifford prefix as learning_material/CHP/examples/ghz.chp through the second H on q0

Run CHP with **silent measurements disabled** (there are none) and **final state** on:

```
.\learning_material\CHP\chp.exe -qs .\reports\chp_vs_stabilizer_assets\circuits_tableau\epr_bell_prep.chp
```

2. EPR Bell prep: CHP vs Python (tableau)

Clifford: $H(0)$, $CNOT(0,1)$ on two qubits in $|00\rangle$.

CHP (-qs), first `printstate` block

Omit the leading banner lines; the tableau is:

```
+ZI
+IX
---
+XX
+ZZ
```

Python (`format_chp_printstate()`)

```
+ZI
+IX

---
+XX
+ZZ
```

The only cosmetic difference is a **blank line** before the separator in Python's formatter; the **Pauli rows match**.

Python explicit **X / Z** bit matrices

CHP does not echo these as numbers; they are the same information as the Pauli grid above.

```
X matrix (4 x 2)
0 0
0 1
1 1
0 0

Z matrix (4 x 2)
1 0
0 0
0 0
1 1
```

CHP second `printstate` (after `gaussian`)

For this Bell state, CHP's second block is **identical** to the first; the ket expansion is $|00\rangle + |11\rangle$ (two terms).

3. GHZ prep: CHP vs Python (tableau)

Clifford: same as `ghz.chp` through `p 2` and `h 0` (no measurements).

CHP (-qs), first `printstate` block

```
+XII
+IZI
+IIY
----
-YIY
```

```
+IYY
+XZZ
```

Python (`format_chp_printstate()`)

```
+XII
+IZI
+IIY

----
-YIY
+IYY
+XZZ
```

Again, rows match; Python inserts a blank line before the rule.

Python explicit **X** / **Z** bit matrices

```
X matrix (6 x 3)
 1 0 0
 0 0 0
 0 0 1
 1 0 1
 0 1 1
 1 0 0

Z matrix (6 x 3)
 0 0 0
 0 1 0
 0 0 1
 1 0 1
 0 1 1
 0 1 1
```

CHP after **gaussian** (second block)

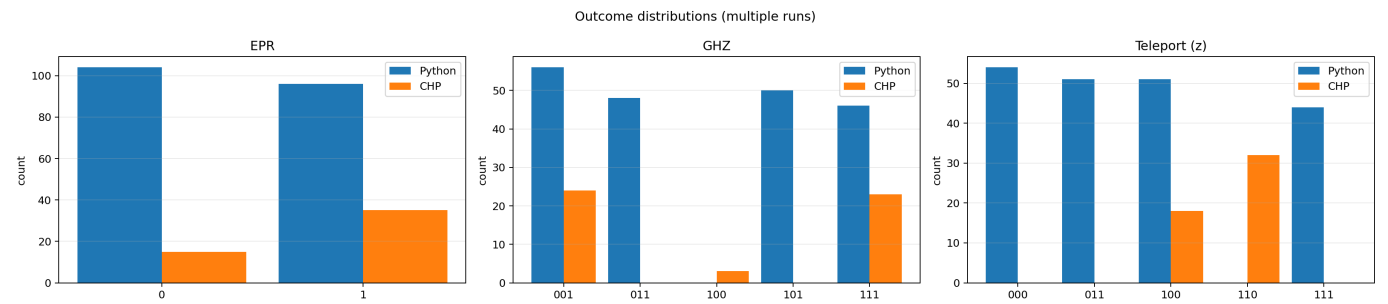
CHP may **change generator presentation** (still the same stabilizer state). Example from the same run (destabilizer row 0 and stabilizer row 2 differ from the first block while the printed ket list remains an equivalent superposition):

```
+XIY
+IZI
+IIY
----
-YIY
+IYY
-ZZX
```

So for **GHZ-style states**, compare Python to CHP's **first** tableau printout for a direct row-by-row match; treat CHP's **second** printout as a canonicalized form.

4. Visual comparison (multiple runs)

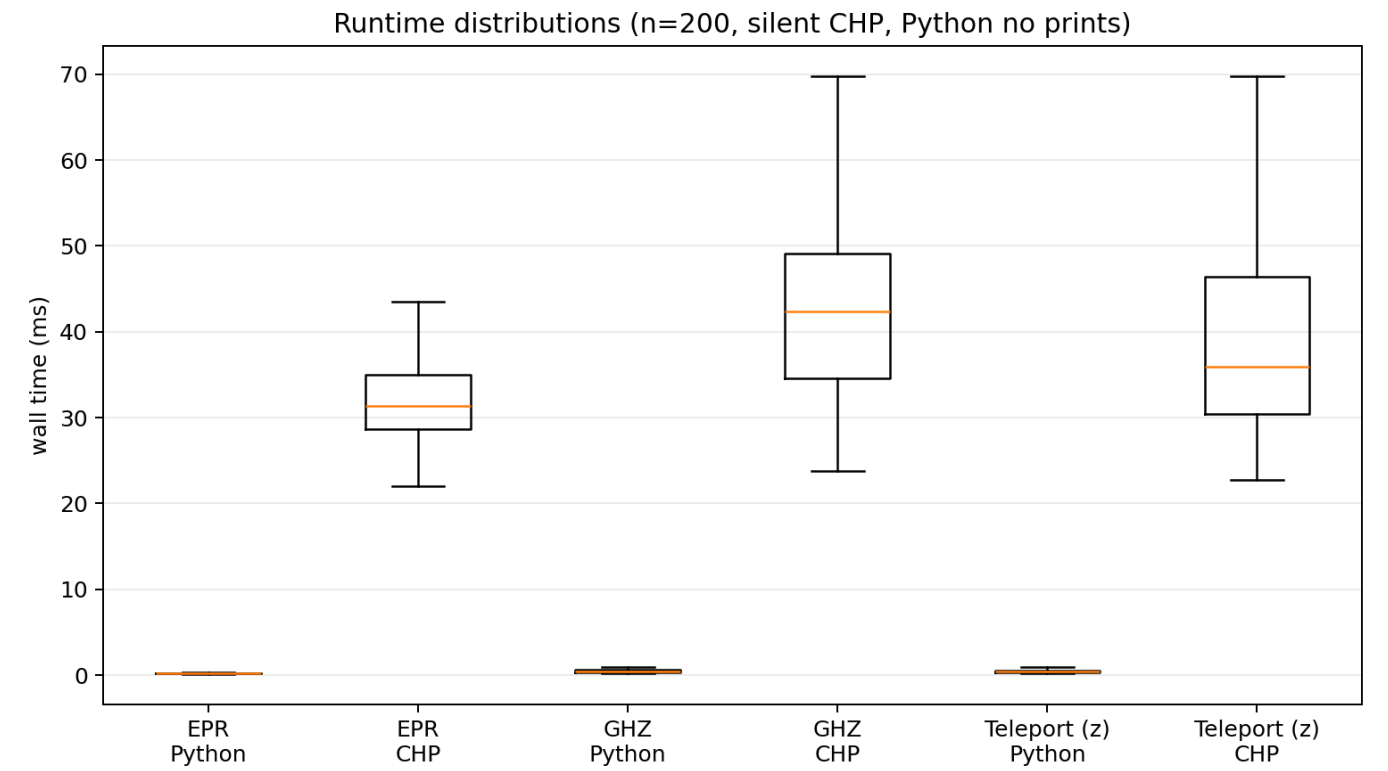
The figure below compares **outcome distributions** across multiple runs for each benchmark circuit, side-by-side (Python vs CHP).



5. Performance metrics (multiple runs)

Runtime distributions

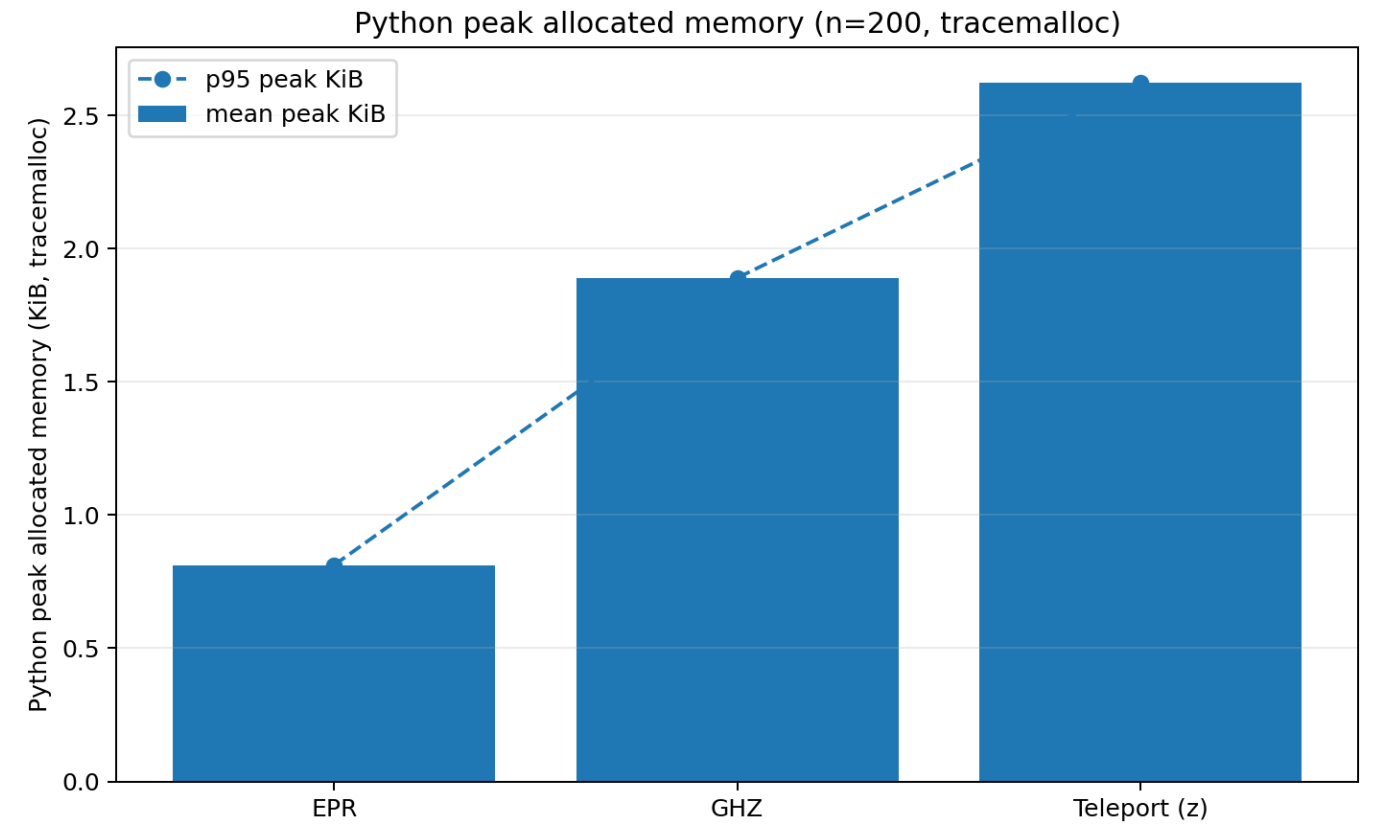
Per-run **wall time** for each circuit and implementation:



Interpretation: CHP is invoked as a **new process** each run (`chp.exe . . .`), so wall time includes **startup and I/O**. Python is timed **in-process** without printing. Use these curves mainly for **variance and relative shape** in this repo setup, not as bare simulator throughput.

Python peak allocated memory

Measured with `tracemalloc` peak allocated bytes during each run:



Raw metrics and auto-summary

Artifact	Path
Full JSON	reports/chp_vs_stabilizer_assets/metrics.json
Short markdown summary	reports/chp_vs_stabilizer_assets/metrics_summary.md

Snapshot from the current `metrics.json` in the repo (timing: **200** runs per side; CHP outcome frequencies from **50** runs):

Circuit	Python p50 (ms)	CHP p50 (ms)	Ratio Python/CHP p50
EPR	0.175	31.305	~0.01×
GHZ	0.419	42.388	~0.01×
Teleport (z)	0.417	35.937	~0.01×

Re-run the benchmark to refresh numbers; the JSON is the source of truth.

6. Benchmark circuits: measurement outcomes (original side-by-side)

The following sections keep the **original** outcome-oriented comparison (single Python sample vs two CHP samples) for the three benchmark circuits. They complement the **deterministic** tableau snapshots above (which stop before measurement).

Circuit Graph 1: EPR

```
flowchart LR
    A["q0: |0>"] --> H0["H(q0)"]
    H0 --> C01["CNOT(q0->q1)"]
    C01 --> M1["MZ(q1)"]
    B["q1: |0>"] --> C01
```

Side-by-side outputs

Python (1 run)

--

```
PY_EPR_M: [1]
```

CHP run A

```
Outcome of measuring qubit 1: 1 (random)
2^0 nonzero basis states
+|11>
```

CHP run B

```
Outcome of measuring qubit 1: 0 (random)
2^0 nonzero basis states
+|00>
```

Comparison note

- Python and CHP both show random collapse branches for Bell measurement (0 or 1).

Circuit Graph 2: GHZ

```
flowchart TB
    subgraph ghz["GHZ circuit (matches ghz.chp)"]
        A["H(0)"] --> B["H(1)"]
        B --> C["CNOT(0,2)"]
        C --> D["CNOT(1,2)"]
        D --> E["P/S(0,1,2)"]
        E --> F["H(0)"]
        F --> M0["MZ(0)"]
        M0 --> G["H(1)"]
        G --> M1["MZ(1)"]
        M1 --> M2["MZ(2)"]
    end
end
```

Side-by-side outputs

Python (1 run)

```
PY_GHZ_M: [1, 0, 0]
```

CHP run A

```
Outcome of measuring qubit 0: 0 (random)
Outcome of measuring qubit 1: 0 (random)
Outcome of measuring qubit 2: 1
2^0 nonzero basis states
+|001>
```

CHP run B

```
Outcome of measuring qubit 0: 1 (random)
Outcome of measuring qubit 1: 1 (random)
Outcome of measuring qubit 2: 1
2^0 nonzero basis states
+|111>
```

Comparison note

- GHZ measurement order and classical processing can differ across implementations; both runs exhibit branch-dependent outcomes.
- CHP shows distinct random branches between run A and run B.

Circuit Graph 3: Teleportation (z input)

CHP file reference: [learning_material/CHP/examples/teleport.chp](#)

```
flowchart LR
    Q0["q0: input z"] --> C01["CNOT(q0->q1)"] --> H0["H(q0)"] --> M0["MZ(q0)"] --> C03["CNOT(q0->q3)"]
    Q1["q1: |0>"] --> H1["H(q1)"] --> C12["CNOT(q1->q2)"] --> C01 --> M1["MZ(q1)"] --> C14["CNOT(q1->q4)"]
    Q2["q2: |0>"] --> C12 --> C42["CNOT(q4->q2)"] --> H2a["H(q2)"] --> C32["CNOT(q3->q2)"] --> H2b["H(q2)"] -->
    M2["MZ(q2)"]
    Q3["q3: anc"] --> C03 --> C32
    Q4["q4: anc"] --> C14 --> C42
```

Side-by-side outputs**Python (1 run, equivalent gate sequence)**

```
PY_TELEPORT_M: [0, 1, 1]
```

CHP run A

```
Outcome of measuring qubit 0: 0 (random)
Outcome of measuring qubit 1: 0 (random)
Outcome of measuring qubit 2: 0
2^0 nonzero basis states
+|0000>
```

CHP run B

```
Outcome of measuring qubit 0: 1 (random)
Outcome of measuring qubit 1: 1 (random)
Outcome of measuring qubit 2: 0
2^0 nonzero basis states
+|11011>
```

Comparison note

- CHP runs A and B both end with $q2 = 0$ for input z in these samples.
- The archived Python sample gave $[0, 1, 1]$ for the three measured bits, which disagrees with both CHP branches on the **last** bit; the benchmark's [metrics.json](#) outcome histograms should be used for up-to-date statistics. A **deterministic** tableau-by-tableau check for teleport is omitted here because the benchmark Python circuit is a simplified wiring and may not match CHP's full teleporter line-for-line.

7. Takeaways

- **Tableau / bit matrices:** For **EPR Bell prep** and **GHZ prep** (prefix circuits in [circuits_tableau/](#)), CHP's **first printstate** block and stabilizer-python's [format_chp_printstate\(\)](#) agree on every **Pauli letter** per row; Python additionally prints the **X and Z binary matrices** for inspection.
- **CHP's second tableau print** may differ after [gaussian\(\)](#) while representing the **same** stabilizer state; Python does not duplicate that second canonicalization step in its formatter.
- **Performance:** See section 5 and [metrics.json](#) / [metrics_summary.md](#); CHP subprocess timing is dominated by **process overhead** in this harness.
- **Teleportation** remains the main area where **measurement-outcome statistics** and single-run samples should be checked against CHP using fresh benchmark output, not only tableau snapshots.